Convex Combination (Weighted Fusion), Worked Out by Hand

Once two retrievers' scores share a scale, you blend them with a single weight α: α of the keyword score plus $(1-\alpha)$ of the semantic score. The blend, and how the final ranking swings with α, computed by hand.

What this document does. It takes the min-max-normalised BM25 and dense scores from PDF #19 and fuses them with a convex combination, sweeping the weight α to show how the merged ranking shifts from semantic-led to keyword-led. It explains why the weights must sum to one and how to tune α. Numbers from Appendix A.

Prerequisite: min-max normalization (#19) — convex combination is meaningless on un-normalised scores. Contrast with RRF (#21), which needs no scores.

1 The formula

Given two normalised score lists $\tilde{s}^{(1)}$ and $\tilde{s}^{(2)}$ over the same documents, the fused score is a weighted average:

$$f_i = \alpha\,\tilde{s}_i^{(1)} + (1-\alpha)\,\tilde{s}_i^{(2)}, \qquad \alpha \in [0,1]$$

The weights α and $1-\alpha$ are non-negative and sum to 1 — that is what makes it *convex*. The consequence: f_i stays inside $[0,1]$ (the same range as the inputs), so the fused scores remain interpretable and no retriever can blow up the scale. α is the single "how much do I trust keywords vs. meaning?" dial.

> **Intuition**
>
> Convex combination is a mixing board with one slider. All the way to $\alpha = 1$ you hear only BM25 (pure keyword); all the way to $\alpha = 0$ only the dense retriever (pure semantic); in between, a blend. Because the two channels were first normalised to the same loudness (#19), the slider controls *balance*, not which channel happens to be louder. Slide until the blend ranks your validation queries best.

2 Worked by hand

The normalised scores from PDF #19:

	A	B	C	D	E
$\tilde{s}^{\text{BM25}}$	1.0000	0.3780	0.7927	0.0000	0.2073
$\tilde{s}^{\text{dense}}$	0.5000	1.0000	0.3571	0.9286	0.0000

Balanced blend, $\alpha = 0.5$. Average the two rows. For A: $0.5(1.0) + 0.5(0.5) = 0.75$. For C: $0.5(0.7927) + 0.5(0.3571) = 0.5749$.

$$f^{0.5} = (\,A\,0.7500,\ B\,0.6890,\ C\,0.5749,\ D\,0.4643,\ E\,0.1036\,) \ \Rightarrow\ A > B > C > D > E.$$

Sweep α and watch the order move.

α	A	B	C	D	E	ranking
0.3 (semantic-led)	0.650	0.813	0.488	0.650	0.062	$B > A{=}D > C > E$
0.5 (balanced)	0.750	0.689	0.575	0.464	0.104	$A > B > C > D > E$
0.7 (keyword-led)	0.850	0.565	0.662	0.279	0.145	$A > C > B > D > E$

At $\alpha = 0.3$ the dense retriever's favourite B wins; at $\alpha = 0.7$ BM25's favourite A dominates and C (a keyword-strong doc) climbs over B. The single weight visibly steers the whole list.

Mini-example · fusion finds what neither retriever ranked first

Document A is BM25's #1 but only dense's #3; document B is dense's #1 but only BM25's #3. At the balanced $\alpha = 0.5$, A wins overall (0.75) because it is *strong in both* (1.0 and 0.5), while B (0.38 and 1.0) is strong in only one. **Convex combination rewards consensus** — documents both retrievers like rise, one-sided favourites sink. That cross-checking is exactly why hybrid beats either retriever alone.

3 Choosing α

- **Start at** $\alpha = 0.5$ and adjust against a labelled query set using nDCG (#18) or MRR (#29) — never by hand-picking a few queries.
- **Shift toward keywords** ($\alpha > 0.5$) when exact matches matter — code, identifiers, names, rare jargon (the `CVE-2024-31082` case). Shift toward semantics ($\alpha < 0.5$) for natural-language, paraphrase-heavy queries.
- **Know the weakness:** convex combination inherits min-max's fragility (#19) — an outlier score distorts the normalisation that feeds it. When that bites, switch to rank-based RRF (#21), which discards score magnitudes altogether.

Equation cheat-sheet

$$f_i = \alpha\,\tilde{s}_i^{(1)} + (1-\alpha)\,\tilde{s}_i^{(2)}, \quad \alpha \in [0,1], \quad f_i \in [0,1]$$

$\alpha{=}1 \Rightarrow$ pure retriever 1, $\alpha{=}0 \Rightarrow$ pure retriever 2, rewards documents strong in *both*

Appendix A · Reference implementation

```
nb = {"A":1.0,"B":0.378,"C":0.7927,"D":0.0,"E":0.2073}     # norm BM25  (#19)
nd = {"A":0.5,"B":1.0,"C":0.3571,"D":0.9286,"E":0.0}       # norm dense (#19)

def fuse(alpha):
    return {k: round(alpha*nb[k] + (1-alpha)*nd[k], 4) for k in nb}

fuse(0.5)   # A 0.75  B 0.689  C 0.5749  D 0.4643  E 0.1036  -> A>B>C>D>E
fuse(0.3)   # B 0.813 wins (semantic-led)
fuse(0.7)   # A 0.85 dominates, C climbs over B (keyword-led)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0